

A Review: Decidable vs. Recognizable Languages

Recall the following languages from last week:

ACC: $\{ \langle M, w, k \rangle \mid M \text{ accepts } w \text{ in } \leq k \text{ steps} \}$

H: $\{ \langle M, w, k \rangle \mid M \text{ halts on } w \text{ in } \leq k \text{ steps} \}$

and

A_{TM} : $\{ \langle M, w \rangle \mid M \text{ is a TM that accepts the input } w \}$

HALT: $\{ \langle M, w \rangle \mid M \text{ is a TM that halts on the input } w \}$

We argued that we can solve ACC, but not A_{TM} . So what's the difference between these languages?

A Review: Decidable vs. Recognizable Languages (2)

When we see an instance of ACC, we have a time limit k . If M accepts w , but takes more than k steps to do it, it is still a no-instance.

If we want to test whether an M every accepts a w (i.e., if $\langle M, w \rangle \in A_{TM}$) we have to test every k :

Does M accept w in one step?

Does M accept w in two steps?

Does M accept w in three steps?

...

Does M accept w in n steps?

...

A Review: Decidable vs. Recognizable Languages (3)

This means that the program

M_A = “On input $\langle M, w \rangle$:

- ① Run M on w .
- ② If it accepts, *accept*.
- ③ *Reject*.

“recognizes” A_{TM} — if an input is in A_{TM} , M_A can eventually verify it for us.

- Problem: if the answer is no, there’s no point where we can stop and be sure of the answer.

So A_{TM} is, in a sense, *half-decidable*.

- If we can build a TM that recognizes the *yes*-instances of a language, like we can for A_{TM} , then we call the language **recognizable**.

*We also call the TM a **recognizer**.*

- There’s a dual concept, too: If $\bar{L}$ is recognizable, then L is **co-recognizable**.

Recognizable Languages in Set Notation

- Recall how we wrote about decidable languages in set notation:

$$L = \{ \langle \text{instance} \rangle \mid P(\langle \text{instance} \rangle) \}$$

where P is a decision problem that can be decided using TMs.

- So far this observation is a bit circular, but it becomes interesting when we try writing recognizable languages in the same way.
- For example, we can write A_{TM} as

$$A_{\text{TM}} = \{ \langle M, w \rangle \mid \exists k, \langle M, w, k \rangle \in \text{ACC} \}$$

- The statement “ $\langle M, w, k \rangle \in \text{ACC}$ ” is decidable. The difference is the quantifier.

Recognizable Languages in Set Notation (2)

It turns out that every recognizable language can be written this way:

- A language L is recognizable iff it can be written in the form

$$L = \{ \langle \text{instance} \rangle \mid \exists \langle \text{something} \rangle, P(\langle \text{instance}, \text{something} \rangle) \}$$

*We'll call the "something" a **certificate**.*

- The difference is a single existential quantifier $\exists$.

Some examples:

$$A_{TM}: \{ \langle M, w \rangle \mid \exists k, \langle M, w, k \rangle \in ACC \}$$

$$HALT: \{ \langle M, w \rangle \mid \exists k, \langle M, w, k \rangle \in H \}$$

$$SQ: \{ \langle x \rangle \mid \exists y \in \mathbb{N}, x = y^2 \}$$

$$SAT: \{ \langle \phi \rangle \mid \phi \text{ is a Boolean formula, and } \exists \tau, \phi(\tau) = \text{TRUE} \}$$

Note: Every decidable language is recognizable.

Recognizable Languages

To see why the two definitions of recognizable languages are equivalent:

We'll give a high-level argument.

- Suppose that a language L is recognized by the TM R . Then,

$$L = \{ \langle x \rangle \mid \exists k, R \text{ accepts } \langle x \rangle \text{ in } k \text{ steps} \}$$

- Suppose $L = \{ \langle \text{instance} \rangle \mid \exists \langle \text{certificate} \rangle, P(\langle \text{instance}, \text{certificate} \rangle) \}$. Then we can build a recognizer R that loops over all of the possible certificates and checks them with a decider for P .

Of course, in order to loop over all of the certificates we need to be sure that we can loop over the certificates. This might be tricky if they aren't numbers. So we'll need to make use of something called an **effective enumeration**.

Effective Enumerations

In order to loop over things like the set of all finite strings or the set of TMs:

We can loop over a set iff we can write a looping program that will print out the elements of that set.

For example, we can write a program that will print out

$$\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots$$

Every finite string in $\{0, 1\}^*$ will eventually be printed by this loop. This sort of loop is called an **effective enumeration**.

I've just showed you an effective enumeration of the strings (over $\{0, 1\}$), and I'll say without proof that there's an effective enumeration over the TM descriptions $\langle M \rangle$.

You can use these enumerations in your programs. If you want to enumerate other sets, you have to describe the enumeration.

Using Effective Enumerations

Consider the following program:

Let $w_0, w_1, \dots$ be an effective ordering of the strings,
and $\langle M_0 \rangle, \langle M_1 \rangle, \dots$ be an effective enumeration of the TMs.

$M =$ “Ignore input:

- ① For $k = 0, 1, 2, \dots$:
- ② For $i = 0$ to k :
- ③ For $j = 0$ to k :
- ④ Run M_i on w_j for k steps.
- ⑤ If it accepts, print $\langle M_i, w_j \rangle$ and continue.”

This program prints exactly the strings in A_{TM} .

This sort of program is called an **enumerator**.

- *A program is recognizable iff it has an enumerator.*

Dovetailing

This program has another nice property:

Let $w_0, w_1, \dots$ be an effective ordering of the strings,
and $\langle M_0 \rangle, \langle M_1 \rangle, \dots$ be an effective enumeration of the TMs.

$M =$ “Ignore input:

- ❶ For $k = 0, 1, 2, \dots$:
- ❷ For $i = 0$ to k :
- ❸ For $j = 0$ to k :
- ❹ Run M_i on w_j for k steps.
- ❺ If it accepts, print $\langle M_i, w_j \rangle$ and continue.”

You’ll notice that what we want to do for this sort of program is nest multiple infinite loops. But if we did that we’d get stuck on the first iteration of the outer loop — the inner loop would never halt.

We fix this problem by putting in a single outer loop that goes to infinity, and bounding the inner loops by the index of the outer one.

This process is called **dovetailing**.

Getting Back to the Language Classes...

We say that a language L is **co-recognizable** iff its complement $\bar{L}$ is recognizable.

Equivalent definitions:

- There is a program R that recognizes $\bar{L}$:
 - If $x \notin L$ (i.e., $x \in \bar{L}$) then R accepts x .
 - If $x \in L$ (i.e., $x \notin \bar{L}$) then R rejects or loops on x .
- There is a program R' that “co-recognizes” L :
 - If $x \notin \bar{L}$ then R' rejects x .
 - If $x \in \bar{L}$ then R' accepts or loops on x .

R' just flips the answer for R .
- There is a decidable predicate Q such that

$$L = \{ \langle x \rangle \mid \forall y, Q(x, y) \}.$$

Relations Between These Classes

We now have three classes of languages: the decidable languages, the recognizable languages, and the co-recognizable languages.

How do these classes relate to each other?

- If we can decide a language (figure out all of its *yes* and *no*-instances), then we can figure out all of its *yes*-instances.
So every decidable language is recognizable.
- The example of A_{TM} shows us that the converse is not true, though.
- If we can decide a language (figure out all of its *yes* and *no*-instances), then we can figure out all of its *no*-instances.
So every decidable language is co-recognizable.
- The converse is also not true.

But, if a language L is *both* recognizable and co-recognizable, it must be decidable.

Why? Dovetailing!

Suppose some TM M_1 that recognizes L and another TM M_2 that recognizes $\bar{L}$. Then the following TM decides L :

M = “On input w :

- ① For $k = 0, 1, 2, \dots$:
- ② Run M_1 on w for k steps.
- ③ If it accepts, *accept*.
- ④ Run M_2 on w for k steps.
- ⑤ If it accepts, *reject*.

If $w \in L$, then M_1 eventually halts, and so does M . If $w \notin L$, then M_2 eventually halts, and so does M .

Another Undecidable Language

We don't want to go through diagonalization every time we get a computability result, so we'll need more tools.

Now that we have one undecidable language, though, we can make use of a tool called a **reduction**:

- A reduction is a program P that relates one decision problem to another.
- The purpose of this program will be to transform one type of problem into another.

We'll use this basic idea to show that the *halting problem* is undecidable.

$$\text{HALT} = \{ \langle M, w \rangle \mid M \text{ halts on input } w \}$$

HALT is Undecidable

Suppose I have an instance $\langle M, w \rangle$ of A_{TM} . Consider what happens if I run it through the following code:

$P =$ “On input $\langle M, w \rangle$:

- ❶ Let $M' =$ “On input x :
 1. Run M on x .
 2. If it accepts, *accept*.
 3. Otherwise, *loop*.”
- ❷ Output $\langle M', w \rangle$.”

So I started with an instance $\langle M, w \rangle$ of A_{TM} , and now I have another instance $\langle M', w \rangle$. Clearly, if M accepts w then so does M' , and if it doesn't accept then neither does M' . But M' also has the property that if it doesn't accept, it loops forever. It never rejects.

Now, this means that M' halts on w iff M accepts w . That is, $\langle M', w \rangle \in \text{HALT}$ iff $\langle M, w \rangle \in A_{TM}$.

HALT is Undecidable (2)

Altogether, this means that if we had a TM M_H that could decide HALT, then we could decide A_{TM} as well: given an instance $\langle M, w \rangle$, we could:

- Run it through P to get $\langle M', w \rangle$,
- run $\langle M', w \rangle$ through M_H , and then
- output the result we get from M_H .

This is the idea of a reduction.

Definition of a **mapping reduction**: Given any two languages A and B , we say $A \leq_m B$ (A mapping reduces to B) if we have some program P such that:

- P halts on all inputs.
- If $x \in A$, then $P(x) \in B$.
- If $x \notin A$, then $P(x) \notin B$.

In the above example, $A = A_{TM}$ and $B = \text{HALT}$.

How Can We Use Mapping Reductions?

If I can solve B , then I can use P to solve A :

- If $P(x) \in B$, then I know that $x \in A$.
- If $P(x) \notin B$, then I know that $x \notin A$.

In fact:

- If I can prove to you that $P(x) \in B$, then I can also prove that $x \in A$.
- If I can prove to you that $P(x) \notin B$, then I can also prove that $x \notin A$.

Remember that if we can solve B then it is decidable. If I can prove to you that something is in B then it is recognizable. That's why I'm making the distinction.

Some Theorems

This means:

Theorem 5.22:

If $A \leq_m B$ and B is decidable, then A is decidable.

Corollary 5.23:

If $A \leq_m B$ and A is undecidable, then B is undecidable.

Theorem 5.28:

If $A \leq_m B$ and B is recognizable, then A is recognizable.

Corollary 5.29:

If $A \leq_m B$ and A is not recognizable, then neither is B .

Some More Theorems

This means:

- If $A \leq_m B$, then $\overline{A} \leq_m \overline{B}$.

Just walk through the definitions and you'll see that the same program P will work for both.

Applying this idea to Theorem 5.28 and Corollary 5.29 from Sipser, we see that

- If $A \leq_m B$ and B is co-recognizable, then A is co-recognizable.
- If $A \leq_m B$ and A is not co-recognizable, then neither is B .

Writing Mapping Reductions

You may have already done this, actually — the PCP question from tutorial 2 was a mapping reduction from PATH to the PCP!

But to start you off we'll give a default reduction from HALT (or $\overline{\text{HALT}}$):

DEFAULT PROOF FORMAT for proving that $\text{HALT} \leq_m B$:

- Consider the TM P “On input $\langle M, w \rangle$ (i.e., the input to HALT):
 - Construct the TM M_0 as follows:
 - $M_0 =$ “On input $\langle \text{Input} \rangle$:
 - $\langle \text{Step 1} \rangle$
 - Run M on w .
 - $\langle \text{Step 3} \rangle$ ”
 - Return $\langle M_0 \rangle$.
- Proof that $\langle M_0 \rangle \in B$ iff $\langle M, w \rangle \in \text{HALT}$.

If $A \neq \text{HALT}$ or $\overline{\text{HALT}}$ you'll need a different template for M_0 .

Examples

That was all a bit abstract, so let's go over a few examples:

Note: $L(M)$ is the set of strings that M accepts, i.e.,

$$L(M) = \{x \in \Sigma^* \mid M \text{ accepts } x\}$$

- $E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$
- $L_1 = \{\langle M \rangle \mid \exists \text{ a TM } N \text{ such that } M \text{ accepts } \langle N \rangle \text{ and } N \text{ accepts } \varepsilon\}$
- $L_2 = \{\langle M \rangle \mid L(M) = \{1\}\}$
- $\text{ALL}_{\text{TM}} = \{\langle M \rangle \mid M \text{ accepts every input}\}$

Are these languages recognizable, co-recognizable, both or neither (no proofs needed yet)?

E_{TM} : An Intuition

I'm going to argue that E_{TM} is co-recognizable.

- If I gave you a program $\langle M \rangle$ and claimed that it was *not* in E_{TM} , that would mean that it is not in the set of TMs that accept no inputs.

So I'd be claiming that it accepts some input x .

- If I told you this x , that might convince you...
- But if I gave you the wrong x , M might or might not loop on it. It could take an infinite time to tell if I'd given you the right x .
- Solution: I tell you both x and how long to run M (number of steps k).

certificate: $\langle x, k \rangle$

On the other hand, E_{TM} is *probably* not recognizable.

- $\overline{A_{TM}}$ isn't recognizable. If I can't prove to you that M fails to accept a *single* given input w , then how could I do so for *every* input?

L_1 : An Intuition

Can you think of a way of proving that an input $\langle M \rangle \in L_1$? Why or Why not?

Yes, if I can tell you the TM N and give you the time you need for both M and N to accept, I should be able to convince you that an $\langle M \rangle$ is in L_1 .

Can you think of a way of proving that an input $\langle M \rangle \notin L_1$? Why or Why not?

No, in order to prove that an $\langle M \rangle \notin L_1$, I have to show for all N that either M does not accept $\langle N \rangle$ or that N does not accept ϵ . But if we can't recognize $\overline{A_{TM}}$, we should expect that proving that a TM doesn't accept a given input is in general hard.

L_2 : An Intuition

Can you think of a way of proving that an input $\langle M \rangle \in L_2$? Why or Why not?

*No, in order to prove that an $\langle M \rangle \in L_2$, I have to prove both that M accepts the input 1 (which I can do) and prove that M does not accept any other string ($\epsilon, 0, 00, 01, 10, 11, \dots$). Again, if we can't recognize $\overline{A_{TM}}$, we should expect that proving that a TM doesn't accept **all** of these inputs is also in general hard.*

Can you think of a way of proving that an input $\langle M \rangle \notin L_2$? Why or Why not?

No, in order to prove that an $\langle M \rangle \notin L_2$, I have to prove that either M does not accept 1 or that there is some other string that it does accept. The second I might be able to do – if M accepts some other string x then I can tell you x and also how long to run it. But what if M accepts no input? That would amount to being able to recognize E_{TM} . But if we can't recognize E_{TM} , we should expect that proving that a TM doesn't accept a given input is in general hard.

ALL_{TM}: An Intuition

Can you think of a way of proving that an input $\langle M \rangle \in \text{ALL}_{\text{TM}}$? Why or Why not?

*No, since in order to do so we'd have to prove that M accepts **every** input. If there were some way to limit ourselves to a finite number of inputs we could do that, but an infinite number of inputs we can't.*

Can you think of a way of proving that an input $\langle M \rangle \notin \text{ALL}_{\text{TM}}$? Why or Why not?

No, since we can't in general prove that a TM won't accept a single known input. If we don't know which input M doesn't accept, that doesn't make the job of proving M doesn't accept it any easier.

Another Approach

We can also use set notation as guide to making our guesses.

Idea: *Try to write these languages using only quantifiers and decidable predicates.*

- $E_{TM} = \{ \langle M \rangle \mid \forall w \in \Sigma^*, k \in \mathbb{N}, \langle M, w, k \rangle \notin ACC \}$.
A single “ $\forall$ ” means we expect E_{TM} to be co-recognizable, but we don’t know if it’s recognizable.
- $L_1 = \{ \langle M \rangle \mid \exists \langle N, k \rangle, \langle M, N, k \rangle \in ACC \wedge \langle N, \varepsilon k \rangle \in ACC \}$.
A single “ $\exists$ ” means we expect L_1 to be recognizable, but we don’t know if it’s co-recognizable.
- $L_2 = \{ \langle M \rangle \mid \forall w \neq 1, k, \exists k', \langle M, 1, k' \rangle \in ACC \wedge \langle M, w, k \rangle \notin ACC \}$
 $= \{ \langle M \rangle \mid \exists k', \forall w \neq 1, k, \langle M, 1, k' \rangle \in ACC \wedge \langle M, w, k \rangle \notin ACC \}$
- $ALL_{TM} = \{ \langle M \rangle \mid \forall w \in \Sigma^*, \exists k \in \mathbb{N}, \langle M, w, k \rangle \in ACC \}$
Multiple quantifiers may indicate that language is recognizable, co-recognizable, both, or neither.

A Recognizer for $\overline{E_{TM}}$

We can prove that E_{TM} is co-recognizable by giving a recognizer for its complement:

Use dovetailing for this:

Let $w_0, w_1, \dots$ be an effective enumeration of Σ^* .

Let $M_E =$ “On input $\langle M \rangle$:

1. For $k = 0$ to ∞ :
2. For $i = 0$ to k :
3. Run M on w_i for k steps.
4. If it accepts, *accept*.

Also: Show that M_E accepts $\langle M \rangle$ iff $M \notin E_{TM}$.

How to Show E_{TM} is not Recognizable?

Basic Idea:

- Start with a language we know is not recognizable (e.g., $\overline{A_{TM}}$ or $\overline{HALT}$).
- Assume that E_{TM} is recognizable. Then it has a recognizer.

Alternatively, there is some way of proving that an $\langle M \rangle$ is in E_{TM} .

- Use this recognizer/prover to find a recognizer for $\overline{HALT}$.

That is, write a program that will let us check whether a program is looping.

- Get a contradiction, and conclude that E_{TM} must not be recognizable.

This is where the mapping reduction idea from earlier comes in.

Showing that $\overline{\text{HALT}} \leq_m E_{\text{TM}}$:

Here's the mapping reduction P :

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.

2. Run M on w .

3. Accept.”

2. Return $\langle M_0 \rangle$.”

- Idea: we've started with an instance of $\overline{\text{HALT}}$ (an input of the form $\langle M, w \rangle$)
- If M loops on w then M_0 will always get stuck on step 2, and so it will always loop (never accept).
- If M halts on w then M_0 will always finish step 2 and accept on step 3.

Note that the input x to M_0 is different from the input w for M .

Proving the Mapping Reduction Works

Now we have to prove that $\langle M_0 \rangle \in E_{TM}$ iff $\langle M, w \rangle \in \overline{HALT}$
(i.e., that $\langle M_0 \rangle \in \overline{E_{TM}}$ iff $\langle M, w \rangle \in HALT$):

($\Leftarrow$) Suppose that $\langle M, w \rangle \in HALT$.

Then, for every input x , M_0 reaches line 2.

When it does, M will eventually halt and M_0 will reach line 3 and accept.

So M_0 will accept every input (and therefore will accept at least one input).

$\Rightarrow \langle M_0 \rangle \in \overline{E_{TM}}$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin HALT$.

Then, for every input x , M_0 reaches line 2.

When it does, M will loop, and so M_0 will never reach line 3 and accept.

So M_0 will not accept any input.

$\Rightarrow \langle M_0 \rangle \in E_{TM}$.

Showing that L_1 is Recognizable

Let's do the same for L_1 : How might we build a recognizer for this language?

Let $\langle N_0 \rangle, \langle N_1 \rangle, \dots$ be an effective enumeration of the TMs.

Let $M_{L_1} =$ "On input $\langle M \rangle$:

1. For $k = 0$ to ∞ :
2. For $i = 0$ to k :
3. Run M on N_i for k steps.
4. Run N_i on ε for k steps.
5. If both accept within k steps, *accept*.

Showing that L_1 is Recognizable (2)

If $\langle M \rangle \in L_1$:

$\Rightarrow$ There is an $\langle N \rangle$ and a number k' such that M accepts $\langle N \rangle$ and N accepts ε , both in k' steps.

$\Rightarrow$ There is a number i' and a number k' such that M accepts $\langle N_{i'} \rangle$ and $N_{i'}$ accepts ε , both in k' steps.

$\Rightarrow$ If M_{L_1} reaches iteration $k = \max i', k'$ and $i = i'$ when run on $\langle M \rangle$, then it will accept on line 5.

Note that M_{L_1} can't loop forever inside of any of its loop iterations, so if it runs long enough it will reach this value for k and i . So if it doesn't reach this line it must have already accepted.

$\Rightarrow M_{L_1}$ accepts $\langle M \rangle$.

Showing that L_1 is Recognizable (3)

- If $\langle M \rangle \notin L_1$:

$\Rightarrow$ For all $\langle N \rangle$ and numbers k' , either M does not accept $\langle N \rangle$ in k' steps or N does not ε in k' steps (or both).

$\Rightarrow$ If M_{L_1} is run on $\langle M \rangle$, then for any loop iteration k, i , either M will not accept on line 3 or N_i will not accept on line 4 (or both).

$\Rightarrow M_{L_1}$ will not accept on this loop iteration.

$\Rightarrow$ Since this is true for every loop iteration, M_{L_1} does not accept $\langle M \rangle$.

Showing that L_1 is Not Co-recognizable

The recognizer for this one is tricky, but the mapping reduction is actually easy:

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.
2. Run M on w .
- 3.

2. Return $\langle M_0 \rangle$.”

- Idea: If we pass on step 1 and M loops on w , then M_0 won't accept anything (and in particular, no N s). So it won't be in L_1 .
- If we can make it accept an appropriate N on step 3, we're done!

Showing that L_1 is Not Co-recognizable (2)

Now, let's do the mapping reduction proof:

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.

2. Run M on w .

3. *Accept*

2. Return $\langle M_0 \rangle$.”

Proof that $\langle M_0 \rangle \in L_1$ iff $\langle M, w \rangle \in \text{HALT}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \text{HALT}$.

Then, for every input x , M_0 reaches line 2.

When it does, M will eventually halt and M_0 will reach line 3 and accept.

So M_0 will accept every input.

$\Rightarrow M$ will accept $\langle N \rangle$, where N is the TM:

“On input x : 1. *accept*.”

$\Rightarrow M$ accepts an $\langle N \rangle$ such that N accepts ε .

$\Rightarrow \langle M_0 \rangle \in L_1$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \text{HALT}$.

Then, for every input x , M_0 reaches line 2.

When it does, M will loop, and so M_0 will never reach line 3 and accept.

So M_0 will not accept any input.

$\Rightarrow \langle M_0 \rangle \notin L_1$.

Showing that L_2 is Not Co-recognizable

We've guessed that L_2 is neither recognizable nor co-recognizable.

Non-co-recognizability is easier to prove in this case, so let's do that first.

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.

2. Run M on w .

3.

2. Return $\langle M_0 \rangle$.”

- Idea: If we pass on step 1 and M loops on w , then M_0 won't accept anything (and in particular, it won't accept 1). So it won't be in L_2 .
- How can we fill in step 3?

Showing that L_2 is Not Co-recognizable (2)

Now, let's do the mapping reduction proof:

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input x :

1. Pass.

2. Run M on w .

3. If $x == 1$, *accept*, otherwise *reject*.”

2. Return $\langle M_0 \rangle$.”

Proof that $\langle M_0 \rangle \in L_2$ iff $\langle M, w \rangle \in \text{HALT}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \text{HALT}$.

Then, for every input x , M_0 reaches line 2.

When it does, M will eventually halt and M_0 will reach line 3.

When it does, it will accept iff x is 1.

$\Rightarrow L(M_0) = \{1\}$.

$\Rightarrow \langle M_0 \rangle \in L_2$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \text{HALT}$.

Then, for every input x , M_0 reaches line 2.

When it does, M will loop, and so M_0 will never reach line 3 and accept.

So M_0 will not accept any input.

$\Rightarrow L(M_0) \neq \{1\}$.

$\Rightarrow \langle M_0 \rangle \notin L_2$.

Showing that L_2 is Not Recognizable

This direction is a bit harder:

- If we pass on step 1 and M loops on w , then M_0 doesn't accept any strings (so $L(M_0) \neq \{1\}$).
- We don't want that: we want $L(M_0)$ to be $\{1\}$ iff M loops.
- Idea: in this template, step 2 acts as a *filter*. We always run step 1, but we only get to step 3 if step 2 halts (if M halts on w).
- We can make M_0 accept 1 in step 1, and something else in step 3:

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

- 1.
2. Run M on w .
- 3.

2. Return $\langle M_0 \rangle$.”

Showing that L_2 is Not Recognizable (2)

Now, let's do the mapping reduction proof:

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input x :

1. If $x == 1$, *accept*.
2. Run M on w .
3. *Accept*.”

2. Return $\langle M_0 \rangle$.”

Proof that $\langle M_0 \rangle \in L_2$ iff $\langle M, w \rangle \in \overline{\text{HALT}}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \overline{\text{HALT}}$.

Then M_0 will accept if x is 1, otherwise it will go on to line 2.

When it does, M will loop, and so M_0 will never reach line 3.

So M_0 accepts only the input 1.

$\Rightarrow L(M_0) = \{1\}$.

$\Rightarrow \langle M_0 \rangle \in L_2$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \overline{\text{HALT}}$.

Then M_0 will accept if x is 1, otherwise it will go on to line 2.

When it does, M will halt, and so M_0 will reach line 3.

Upon reaching line 3, M_0 will accept.

So M_0 accepts every input.

$\Rightarrow L(M_0) \neq \{1\}$.

$\Rightarrow \langle M_0 \rangle \notin L_2$.

Showing that ALL_{TM} is Not Co-recognizable

We've guessed that ALL_{TM} is neither recognizable nor co-recognizable. Once again, non-co-recognizability is easier to prove in this case, so let's do that first.

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input x :

1. Pass.

2. Run M on w .

3.

2. Return $\langle M_0 \rangle$.”

- Idea: If we pass on step 1 and M loops on w , then M_0 won't accept anything (and in particular, it won't accept everything). So it won't be in ALL_{TM} .

Showing that ALL_{TM} is Not Co-recognizable (2)

Now, let's do the mapping reduction proof:

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input x :

1. Pass.

2. Run M on w .

3. *Accept*.”

2. Return $\langle M_0 \rangle$.”

Proof that $\langle M_0 \rangle \in \text{ALL}_{\text{TM}}$ iff $\langle M, w \rangle \in \text{HALT}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \text{HALT}$.

Then M_0 always gets to line 2.

When it does, M will halt, and so M_0 will reach line 3 and accept.

So M_0 accepts every input.

$\Rightarrow \langle M_0 \rangle \in \text{ALL}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \text{HALT}$.

Then M_0 always gets to line 2.

When it does, M will loop, and so M_0 will never reach line 3.

So M_0 doesn't accept any input (and so doesn't accept every input, either).

$\Rightarrow \langle M_0 \rangle \notin \text{ALL}_{\text{TM}}$.

Showing that ALL_{TM} is Not Recognizable

This one is actually pretty tricky:

- We want M_0 to accept every string iff M loops on w .
- Last time we said that step 2 acts as a filter: If M loops we don't get to step 3, so any string we would accept at that point is filtered out.
- Idea: We don't want a filter, so we need to change step 2.

Think back to A_{TM} : If M loops, how many k will be valid certificates?

Consider the TM P “On input $\langle M, w \rangle$:

1. Construct the TM M_0 as follows:

M_0 = “On input x :

1. Pass.

- 2.

- 3.

2. Return $\langle M_0 \rangle$.”

Showing that ALL_{TM} is Not Recognizable (2)

Now, let's do the mapping reduction proof:

- Consider the TM P “On input $\langle M, w \rangle$:
 - Construct the TM M_0 as follows:

$M_0 =$ “On input $\langle k \rangle$:
 - Pass.
 - Run M on w for k steps.
 - If it halts, *loop*, otherwise *accept*.”
 - Return $\langle M_0 \rangle$.”

Proof that $\langle M_0 \rangle \in \text{ALL}_{\text{TM}}$ iff $\langle M, w \rangle \in \overline{\text{HALT}}$:

($\Leftarrow$) Suppose that $\langle M, w \rangle \in \overline{\text{HALT}}$.

Then M_0 always gets to line 2.

When it does, M not halt within k steps for any input k , since it loops.

So M_0 accepts every input.

$\Rightarrow \langle M_0 \rangle \in \text{ALL}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M, w \rangle \notin \overline{\text{HALT}}$.

Then M_0 always gets to line 2.

When it does, M will halt for some input k .

For this k , M_0 will loop, and so it will not accept.

$\Rightarrow \langle M_0 \rangle \notin \text{ALL}_{\text{TM}}$.

If There's Time: Proving that $L_2 \leq_m \text{ALL}_{\text{TM}}$

This time we're not reducing from HALT or $\overline{\text{HALT}}$, so we can't use the default reduction.

What can we do instead?

Consider the TM P “On input $\langle M \rangle$:

1. Construct the TM M_0 as follows:

$M_0 =$ “On input $\langle x, k \rangle$:

1. Run M on 1.
2. If it rejects, *loop*. Otherwise,
3. If $x \neq 1$:
4. Run M on x for k steps.
5. If it accepts, *loop*.
6. *Accept*. ”

2. Return $\langle M_0 \rangle$.”

Proof that $\langle M \rangle \in L_2$ iff $\langle M_0 \rangle \in \text{ALL}_{\text{TM}}$:

($\Leftarrow$) Suppose that $\langle M \rangle \in L_2$.

Then M accepts only the input 1.

So M will accept 1 in step 1, and will get to step 2 3, and 4.

When it does, M not accept within k steps for any input $x \neq 1$, since M only accepts 1.

So M_0 accepts every input $\langle x, k \rangle$ on line 6.

$\Rightarrow \langle M_0 \rangle \in \text{ALL}_{\text{TM}}$.

($\Rightarrow$) Suppose that $\langle M \rangle \notin L_2$.

Then M either does not accept 1 or it accepts some other string.

If M does not accept 1, it will loop on either line 1 or 2.

If M does accept 1, then there is some string $x \neq 1$ that it also accepts.

Let k be the number of steps it takes to do so.

On input $\langle x, k \rangle$ M will accept on line 5, and so M_0 will loop.

$\Rightarrow M_0$ will not accept this $\langle x, k \rangle$.

$\Rightarrow \langle M_0 \rangle \notin \text{ALL}_{\text{TM}}$.